Freeze Zone / Freeze State / Torch Item Implementation Guide

D:\FinalProject current codebase - extracted source points and required edits

Core decision

Keep AIceChillSpawner/AIceChillZone as the visible freeze-zone actor flow, but move the real freeze state into AMyCharacter::bFrozen. Add ATorchItem as an ABaseItem subclass. For the final multiplayer version, make GameLogic authoritative for freezing, thawing, and burn damage.

1. System Overview

Area	Current state	Required edit	Reference
Freeze zone spawn	AIceChillSpawner and AIceChillZone already exist.	Local test can use current actors. Final version should let the server decide spawn position and timing.	Map/IceCave/IceChillSpawner.* Map/IceCave/IceChillZone.*
Character freeze state	AMyCharacter has no bFrozen, ApplyFreeze, EndFreeze, or IsFrozen.	Add explicit freeze state, movement lock, input lock, and optional frozen overlay.	D:/Download/MyCharacter.h/.cpp Freeze sections
Server authority	GameLogic::UpdatePlayerMovement computes server position.	Add bFrozen and FreezeRemainTime to Session, then stop server movement while frozen.	session.h GameLogic.cpp
Torch item	BaseItem pickup/drop/equip flow already exists.	Add ATorchItem. Use local DoHit only for prototype; move burn/thaw judgment to server later.	IceCave.zip TorchItem.* BaseItem.*
Item registration	Room::LoadStage registers BuildingStage items only.	Register Torch ItemData for IceCave and match client ItemID.	RoomManager.cpp RoomManager.h

2. Freeze Zone Spawn

2.1 Code to Extract or Reuse

File	Functions / members	Why it matters	Apply to current project
IceChillSpawner.h	ChillZoneClass, SpawnArea, SpawnAccumulator, NextSpawnTime	Controls spawn class, spawn bounds, and interval.	Already present. Tune values in Blueprint or defaults.
IceChillSpawner.cpp	Tick, UpdateSpawner, SpawnChillZones, GetRandomSpawnLocation	Main loop that creates chill zones by phase.	Reusable for local tests.
IceChillZone.h	Radius, FreezeBuildUpTime, ZoneLifeTime, CharacterStayTimes	Tracks how long each character stayed inside the zone.	Already present.
IceChillZone.cpp	OnChillBeginOverlap, OnChillEndOverlap, Tick	Adds/removes characters and triggers freeze after buildup.	Only ApplyFreezeToCharacter needs a direction change.

Current AIceChillZone::ApplyFreezeToCharacter directly disables CharacterMovement and restores it with a timer. That does not work well with TorchItem because there is no IsFrozen state to query. Replace that local movement-only freeze with AMyCharacter::ApplyFreeze().

```
void AIceChillZone::ApplyFreezeToCharacter(AMyCharacter* Character)
{
    if (!IsValid(Character) || Character->IsDead())
    {
        return;
    }

    Character->ApplyFreeze();
```

```
}
```

> **Server-authoritative version**
>
> For final multiplayer, do not let each client randomly freeze players. Let GameLogic own freeze-zone spawn and freeze judgment, then broadcast a FreezeStatePacket. UNetworkInstance should call AMyCharacter::ApplyFreeze() or EndFreeze() after receiving that packet.

2.2 Server Spawn Extension

File	Add	Purpose
GameLogic.h	IceChillZoneData, chillZones, nextChillZoneID, spawn interval/count members	Server owns active chill zones.
GameLogic.cpp	UpdateIceChillZones, SpawnIceChillZones, GetRandomIceChillSpawnLocation	Spawn zones during IceCave phase 2/3.
Packet.h	Reuse SpawnObjectPacket or add a ChillZone packet	All clients see the same zone.
UNetworkInstance.cpp	Spawn local AIceChillZone after receiving server event	Visual-only client representation.

```
struct IceChillZoneData
{
    int zoneID = -1;
    float x = 0.f;
    float y = 0.f;
    float z = 0.f;
    float radius = 650.f;
    float lifeRemainTime = 7.f;
    float freezeBuildUpTime = 2.5f;
    std::unordered_map<int, float> stayTimesBySessionID;
};
```

3. AMyCharacter Freeze State

3.1 MyCharacter.h Required Edits

Location	Edit	Purpose
ERecStatusEffectType	Add Freeze enum value.	Let CanReceiveStatusEffect handle freeze immunity.
Public Status API	Add ApplyFreeze, EndFreeze, IsFrozen.	Called by freeze zones, network handlers, and TorchItem.
Component section	Add FrozenOverlayMeshComponent and mesh/material config.	Optional visual feedback while frozen.
Private section	Add bool bFrozen and UpdateFrozenOverlay.	Store state and refresh visuals.

```
UENUM(BlueprintType)
enum class ERecStatusEffectType : uint8
{
    Slow      UMETA(DisplayName = "Slow"),
    Stun      UMETA(DisplayName = "Stun"),
    Knockback UMETA(DisplayName = "Knockback"),
    Freeze    UMETA(DisplayName = "Freeze"),
};

UFUNCTION(BlueprintCallable, Category = "Status")
void ApplyFreeze();

UFUNCTION(BlueprintCallable, Category = "Status")
void EndFreeze();

UFUNCTION(BlueprintPure, Category = "Status")
bool IsFrozen() const { return bFrozen; }


UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Status|Freeze", meta =
    (AllowPrivateAccess = "true"))
TObjectPtr<UStaticMeshComponent> FrozenOverlayMeshComponent;

UPROPERTY(EditDefaultsOnly, BlueprintReadOnly, Category = "Status|Freeze", meta =
    (AllowPrivateAccess = "true"))
TObjectPtr<UStaticMesh> FrozenOverlayMesh;

UPROPERTY(EditDefaultsOnly, BlueprintReadOnly, Category = "Status|Freeze", meta =
    (AllowPrivateAccess = "true"))
TObjectPtr<UMaterialInterface> FrozenOverlayMaterial;

UPROPERTY(VisibleInstanceOnly, Category = "Status|Freeze")
bool bFrozen = false;

void UpdateFrozenOverlay();
```

3.2 MyCharacter.cpp Required Edits

Function / location	Edit	Reason
include area	Add Components/StaticMeshComponent.h.	Needed for FrozenOverlayMeshComponent.
constructor	Create and hide FrozenOverlayMeshComponent.	Prepare frozen overlay.
CanReceiveStatusEffect	Add Freeze case.	Manipulator immunity should block freeze too.
ApplyRoleStats	If bFrozen, stop movement and set MaxWalkSpeed to 0.	Role/skill updates must not restore speed while frozen.
Move / DoMove / Jump / Attack	Return early if bFrozen.	Block input and attacks while frozen.
ApplyDeadState / ApplyAliveState	Call EndFreeze if bFrozen.	Avoid stale freeze after death or respawn.

```
void AMyCharacter::ApplyFreeze()
{
    if (IsDead() || !CanReceiveStatusEffect(ERecStatusEffectType::Freeze))
    {
        return;
    }

    bFrozen = true;
```

```
    CachedMoveRight = 0.f;
    CachedMoveForward = 0.f;
    StopJumping();

    if (UCharacterMovementComponent* MoveComp = GetCharacterMovement())
    {
        MoveComp->StopMovementImmediately();
        MoveComp->MaxWalkSpeed = 0.f;
    }

    UpdateFrozenOverlay();
}

void AMyCharacter::EndFreeze()
{
    if (!bFrozen)
    {
        return;
    }

    bFrozen = false;
    UpdateFrozenOverlay();
    ApplyRoleStats();
}

void AMyCharacter::ApplyRoleStats()
{
    if (UCharacterMovementComponent* MoveComp = GetCharacterMovement())
    {
        if (bFrozen)
        {
            MoveComp->StopMovementImmediately();
            MoveComp->MaxWalkSpeed = 0.f;
        }
        else
        {
            const float BaseRoleMult = GetRoleSpeedMultiplier(RoleType);
            const float SkillMult = GetRoleSkillSpeedMultiplier();
            MoveComp->MaxWalkSpeed = BaseWalkSpeed * BaseRoleMult * SkillMult;
        }
    }

    // Keep the existing RoleType-specific MaxHP, AttackDamage, and KnockbackCoefficient logic.
}
```

4. Server-Authoritative Freeze

The server already computes movement in GameLogic::UpdatePlayerMovement. If only the client is frozen, server correction can move the player again. Add freeze state to Session and block server movement while frozen.

File	Edit	Why
session.h	Add bool bFrozen and float FreezeRemainTime.	Store per-player freeze state.
GameLogic.h	Declare ApplyFreezeToPlayer, EndFreezeOnPlayer, BroadcastFreezeState.	Centralize server API.
GameLogic.cpp	Handle bFrozen early in UpdatePlayerMovement.	Stop server-side position and velocity updates.
Packet.h	Add PKT_S2C_FREEZE_STATE_BRD and FreezeStatePacket.	Synchronize client visuals.
UNetworkInstance.cpp	Add HandleFreezeStateChanged.	Call ApplyFreeze/EndFreeze on the matching character.

```
// session.h - inside Session
bool bFrozen = false;
float FreezeRemainTime = 0.0f;

// Packet.h example
PKT_S2C_FREEZE_STATE_BRD = 125,

struct FreezeStatePacket
{
    int32_t targetID;
    int32_t bFrozen;  // 0=false, 1=true
    float duration;
};
```

```
void GameLogic::ApplyFreezeToPlayer(int sessionID, float duration)
{
    auto it = players.find(sessionID);
    if (it == players.end() || !it->second)
    {
        return;
    }

    Session* target = it->second;
    GameData& gd = target->gameDatas;
    if (!gd.isConnected || gd.characterState != 0)
    {
        return;
    }

    {
        std::lock_guard<std::mutex> lock(target->MoveIntentLock);
        target->bFrozen = true;
        target->FreezeRemainTime = duration;
        target->HorizontalVelocityX = 0.0f;
        target->HorizontalVelocityY = 0.0f;
    }

    BroadcastFreezeState(target->playerUID, 1, duration);
}
```

5. Torch Item

5.1 Files and Code to Extract

Target	Extract	Required adjustment
Map/IceCave/TorchItem.h	ATorchItem class, FTorchBurnState, burn/thaw members.	Fix include path for the current project.
Map/IceCave/TorchItem.cpp	DoHit, ApplyBurn, UpdateBurns, TryStartThaw, UpdateThaw.	Use as local prototype first. Move final judgment to server.
BaseItem.h/.cpp	StartUse, DoHit, OwnerCharacter, Damage, KnockbackPower, Range, Radius.	Keep as the base class. TorchItem inherits ABaseItem.
MyCharacter.cpp	AnimNotify_AttackHit calls CurrentItem->DoHit().	Already connected. Torch DoHit will be called here.

Compile dependency

TorchItem calls Target->IsFrozen() and Target->EndFreeze(). AMyCharacter must implement those functions first.

```
void ATorchItem::UpdateThaw(float DeltaTime)
{
    if (!CurrentThawTarget)
    {
        return;
    }

    if (DeltaTime <= 0.f || !IsValidThawTarget(CurrentThawTarget))
    {
        CancelThaw();
        return;
    }

    ThawProgress += DeltaTime;
    if (ThawProgress >= ThawRequiredTime)
    {
        AMyCharacter* ThawedTarget = CurrentThawTarget;
        ThawedTarget->EndFreeze();
        BP_OnUnfreezeTarget(ThawedTarget);
        CancelThaw();
    }
}
```

5.2 Attack and Item Flow Edits

Location	Current issue	Edit direction
AMyCharacter::Attack()	AttackType is calculated, but RequestAttackInput(0) is sent.	Send RequestAttackInput(AttackType).
GameLogic::HandleAttackInput	const int attackType = 0 is hard-coded.	Use gd.equippedItemID != -1 ? 1 : 0, or read AttackPacket.
GameLogic::HandleAttackHitReport	Mostly normal attack damage.	If equipped item is Torch, branch into thaw/burn logic.

```
// AMyCharacter::Attack() direction
const int32 AttackType = CurrentItem ? 1 : 0;
NetworkInstance->RequestAttackInput(AttackType);

// Simple server-side direction
const int attackType = gd.equippedItemID != -1 ? 1 : 0;
```

6. Server Item Registration and Safety Fix

The client Torch actor ItemID must match the server ItemData.ObjectID registered in Room::LoadStage.

File	Edit	Example
RoomManager.cpp	Add Torch ItemData for IceCave stageNum.	ObjectID = 101, ObjectType = WEAPON_ITEM.
BP_TorchItem or ATorchItem instance	Set ItemID to 101.	Matches server ObjectID.
RoomManager.h	Change GetItemData to find-based lookup.	Avoid auto-creating missing item IDs.

```
if (stageNum == 2) // IceCaveStage
{
    ItemData torch{};
    torch.ObjectID = 101;
    torch.ObjectType = e_ObjectType::WEAPON_ITEM;
    torch.x = 0.0f;
    torch.y = 0.0f;
    torch.z = 500.0f;
    torch.ownerUID = -1;
    torch.bEquipped = false;
    m_ItemObjects[torch.ObjectID] = torch;
}

ItemData* GetItemData(int itemID)
{
    auto it = m_ItemObjects.find(itemID);
    if (it == m_ItemObjects.end())
    {
        return nullptr;
    }

    return &it->second;
}
```

7. Recommended Implementation Order

1. Add bFrozen, ApplyFreeze, EndFreeze, IsFrozen, and UpdateFrozenOverlay to AMyCharacter.

2. Change AIceChillZone::ApplyFreezeToCharacter to call Character->ApplyFreeze() for local testing.

3. Verify that entering a chill zone blocks movement, jumping, attacks, and shows the overlay.

4. Add ATorchItem.h/.cpp and create BP_TorchItem based on ATorchItem.

5. Match BP_TorchItem ItemID with server Room::LoadStage ItemData.ObjectID.

6. Verify local thaw: torch hit on a frozen character calls EndFreeze().

7. Add bFrozen and FreezeRemainTime to Session.

8. Block movement in GameLogic::UpdatePlayerMovement while bFrozen is true.

9. Add FreezeStatePacket and call ApplyFreeze/EndFreeze from UNetworkInstance.

10. Move final torch thaw/burn judgment into GameLogic::HandleAttackHitReport.

8. Test Checklist

Area	Check	Pass criteria
Freeze zone	AIceChillSpawner ChillZoneClass is assigned.	BP_IceChillZone spawns.
Freeze zone	ChillCollision overlaps Pawn.	OnChillBeginOverlap is called.
Character state	ApplyFreeze is called after FreezeBuildUpTime.	bFrozen=true, movement stops, overlay appears.
Input lock	Test Move, Jump, and Attack.	Input is ignored while frozen.
Thaw	Call EndFreeze.	bFrozen=false and speed returns through ApplyRoleStats.
Server movement	Run UpdatePlayerMovement while bFrozen.	Server position does not advance.
Item	Register Torch in Room::LoadStage.	GetItemData(101) returns non-null.
Torch	Torch branch in HandleAttackHitReport.	Frozen target calls EndFreezeOnPlayer.

9. Do Not Overwrite

- Do not overwrite the current project MyCharacter.cpp with D:/Download/MyCharacter.cpp.

- Keep current networking flow: PlayAttackMontageFromServer(int32 AttackType, uint32 AttackSeq), ApplyLocalServerCorrection, ApplyEquipItemVisual, and ApplyDropItemVisual.

- Use IceChillZone direct DisableMovement only as a temporary prototype, if at all.

- Treat UGameplayStatics::ApplyDamage in TorchItem.cpp as local prototype logic. Final burn/thaw judgment should run in server GameLogic.

- Final freeze-zone random position must be chosen by the server so every client sees the same zone.